1.Train Fare Calculation System – Python Project

1. Problem Statement

To develop a Python program that calculates fares for multiple passengers based on distance slabs, travel class, age concessions, reservation fees, and cancellation penalties.

2. Objectives

- Calculate fares based on distance.
- Apply travel-class charges.
- Provide age-based concessions.
- Add reservation fees.
- Calculate cancellation penalties and display the final amount.

3. Assumptions

- Distance fare: ₹2 per km.
- Classes: Sleeper (1×), AC (2×).
- Children below 12 years get 50% concession.
- Senior citizens aged 60 and above get 30% concession.
- Reservation fee: ₹50 per passenger.
- Cancellation penalty: 25% of the ticket fare.

4. Algorithm

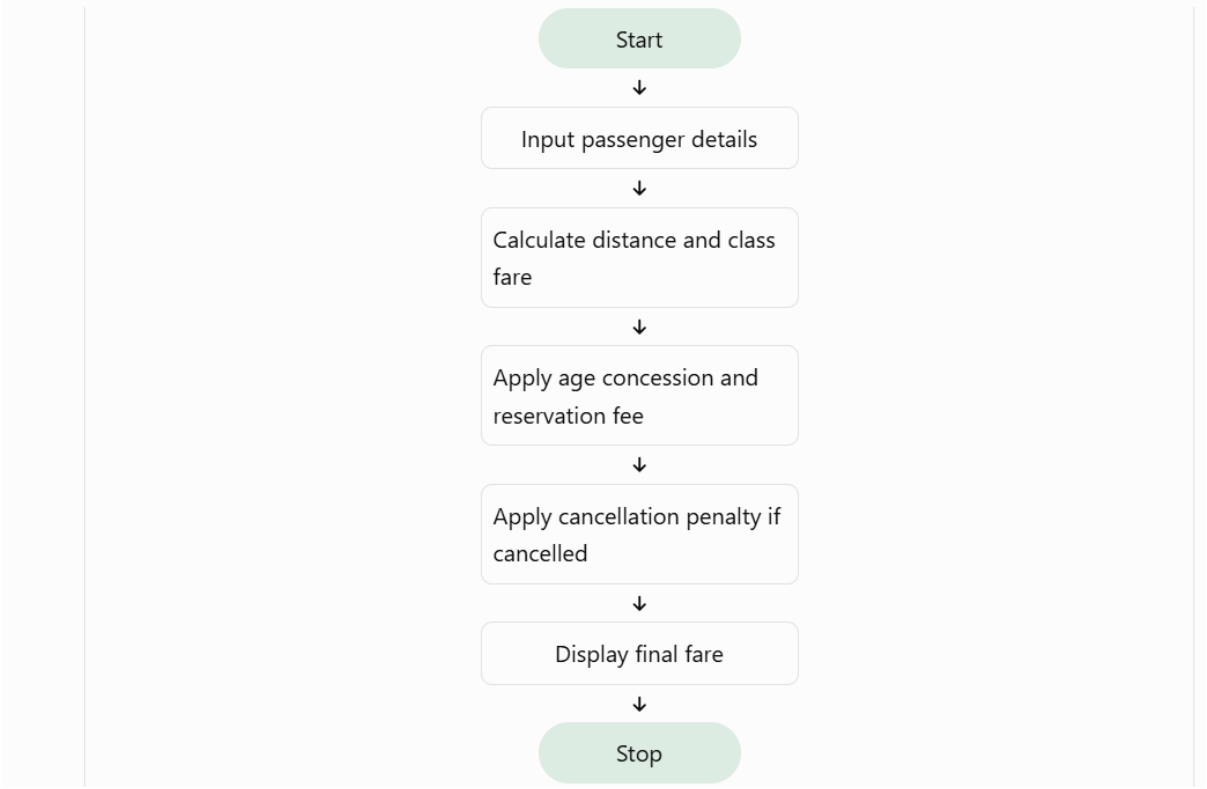
1. Start the program.
2. Input the number of passengers.
3. Enter each passenger's age, distance, and travel class.
4. Calculate the base fare using distance and class.
5. Apply age concession.
6. Add reservation fee.
7. If cancelled, deduct the cancellation penalty.
8. Display the final fare.

9. Stop.

5. Pseudocode

```
START
INPUT number_of_passengers
FOR each passenger:
  INPUT age, distance, travel_class
  fare = distance * 2
  IF travel_class == "AC":
    fare = fare * 2
  IF age < 12:
    fare = fare * 0.50
  ELSE IF age >= 60:
    fare = fare * 0.70
  fare = fare + 50
  INPUT cancellation_status
  IF cancelled:
    fare = fare * 0.25
  DISPLAY fare
END FOR
STOP
```

6. Flowchart



7. Python Source Code

```
n = int(input("Enter number of passengers: "))

for i in range(n):
    print(f"\nPassenger {i + 1}")
    age = int(input("Enter age: "))
    distance = float(input("Enter distance in km: "))
    travel_class = input("Enter class (Sleeper/AC): ").strip().lower()

    fare = distance * 2

    if travel_class == "ac":
        fare *= 2

    if age < 12:
```

```
    fare *= 0.50
elif age >= 60:
    fare *= 0.70

fare += 50

cancelled = input("Cancelled? (yes/no): ").strip().lower()

if cancelled == "yes":
    fare *= 0.25

print(f"Final fare: Rs. {fare:.2f}")
```

8. Test Cases

Age	Distance	Class	Cancelled	Expected fare
25	100 km	Sleeper	No	₹250.00
10	100 km	Sleeper	No	₹150.00
65	100 km	AC	No	₹330.00
30	100 km	AC	Yes	₹112.50

9.Sample Input and Output

```
Passenger 1
Enter age: 56
Enter distance in km: 600
Enter class (Sleeper/AC): AC
Cancelled? (yes/no): NO
Final fare: Rs. 2450.00

Passenger 2
Enter age: 58
Enter distance in km: 700
Enter class (Sleeper/AC): AC
Cancelled? (yes/no): NO
Final fare: Rs. 2850.00

Passenger 3
Enter age: 49
Enter distance in km: 300
Enter class (Sleeper/AC): AC
Cancelled? (yes/no): NO
Final fare: Rs. 1250.00
```

10. Conclusion

The Python program calculates fares for multiple passengers by considering distance, travel class, age concessions, reservation fees, and cancellation penalties. The assumed rates can be modified according to actual railway rules.

2.Patient Billing and Payment Classification System

1. Problem Statement

Develop a Python program to calculate patient bills using consultation, diagnostic, and treatment charges, apply concessions, validate billing constraints, and classify patients based on payment status.

2. Objectives

- Calculate total medical charges.
- Apply eligible concessions.
- Validate billing amounts.
- Classify patients according to payment status.

3. Assumptions

- Consultation fee: ₹500
- Diagnostic charges: ₹1,000
- Treatment charges: ₹2,000
- Concession: 10% if total charges exceed ₹3,000.
- Payment status: Paid, Partial, or Unpaid.
- Negative charges are invalid.

4. Algorithm

1. Start.
2. Input patient details and charges.
3. Validate charges.

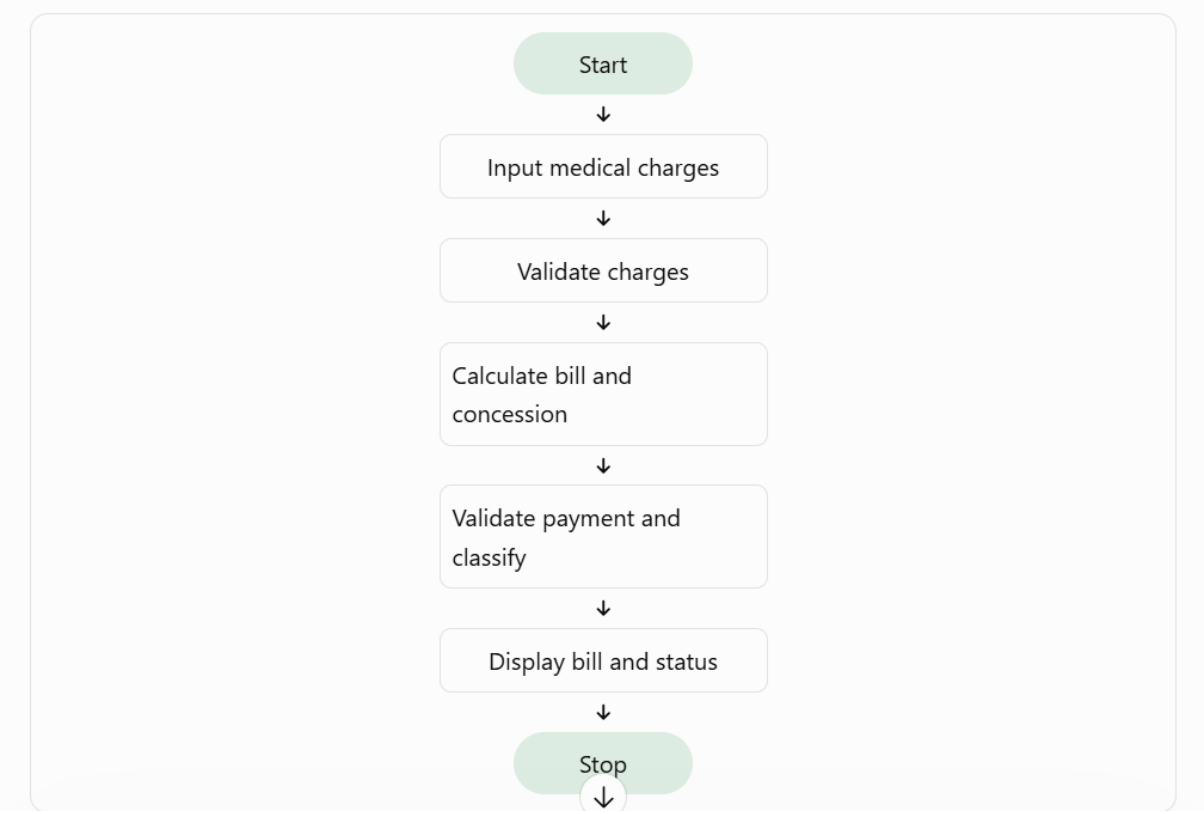
4. Calculate total bill.
5. Apply eligible concession.
6. Input amount paid.
7. Classify payment status.
8. Display final bill.
9. Stop.

5. Pseudocode

```
START
INPUT consultation, diagnosis, treatment
IF any charge < 0:
  DISPLAY Invalid charges
ELSE:
  total = consultation + diagnosis + treatment
  IF total > 3000:
    discount = total * 0.10
  ELSE:
    discount = 0
  bill = total - discount
  INPUT amount_paid
  IF amount_paid < 0 OR amount_paid > bill:
    DISPLAY Invalid payment
  ELSE IF amount_paid == bill:
    status = "Paid"
  ELSE IF amount_paid > 0:
    status = "Partial"
  ELSE:
    status = "Unpaid"
  DISPLAY bill, status
```

STOP

6. Flowchart



7. Python Source Code

```
total = consultation + diagnostic + treatment
  discount = total * 0.10 if total > 3000 else 0
  bill = total - discount

  print("Total bill:", bill)
  paid = float(input("Amount paid: "))

  if paid < 0 or paid > bill:
    print("Invalid payment")
  else:
    if paid == bill:
      status = "Paid"
```

```
elif paid > 0:
    status = "Partial"
else:
    status = "Unpaid"

print("Discount:", discount)
print("Final bill:", bill)
print("Payment status:", status)
```

8. Test Cases

Charges (₹)	Paid (₹)	Expected result
500, 1000, 2000	3150	Bill ₹3150, Paid
500, 1000, 1000	1000	Bill ₹2500, Partial
500, 500, 500	0	Bill ₹1500, Unpaid
-100, 500, 500	0	Invalid charges

10. Sample Output

Consultation charge: 500
Diagnostic charge: 1000
Treatment charge: 400
Total bill: 1900.0
Amount paid: 700
Discount: 0
Final bill: 1900.0
Payment status: Partial

10. Conclusion

The program calculates patient bills, applies conditional concessions, validates charges and payments, and classifies patients as Paid, Partial, or Unpaid.

3.Student Assessment Analysis System – Python Project

1. Problem Statement

Develop a Python program to compare student scores across multiple examinations, calculate improvements and declines, identify weak subjects, and classify students based on their performance.

2. Objectives

- Compare examination scores.
- Calculate score improvements or declines.
- Identify consistently weak subjects.
- Classify students by performance.

3. Assumptions

- Marks are between 0 and 100.
- Two examinations are considered.
- A subject is weak if marks are below 40 in both exams.
- Classification: Excellent (≥ 80), Good (≥ 60), Average (≥ 40), Fail (< 40), based on average marks.

4. Algorithm

1. Start.
2. Input subject names and marks for two exams.
3. Validate marks.
4. Calculate average and score changes.
5. Identify consistently weak subjects.
6. Classify overall performance.
7. Display results.
8. Stop.

5. Pseudocode

START

INPUT subjects and marks for two exams

FOR each subject:

  IF marks are invalid:

    DISPLAY error

  ELSE:

    change = exam2 - exam1

    average = (exam1 + exam2) / 2

    IF both marks < 40:

      mark subject as weak

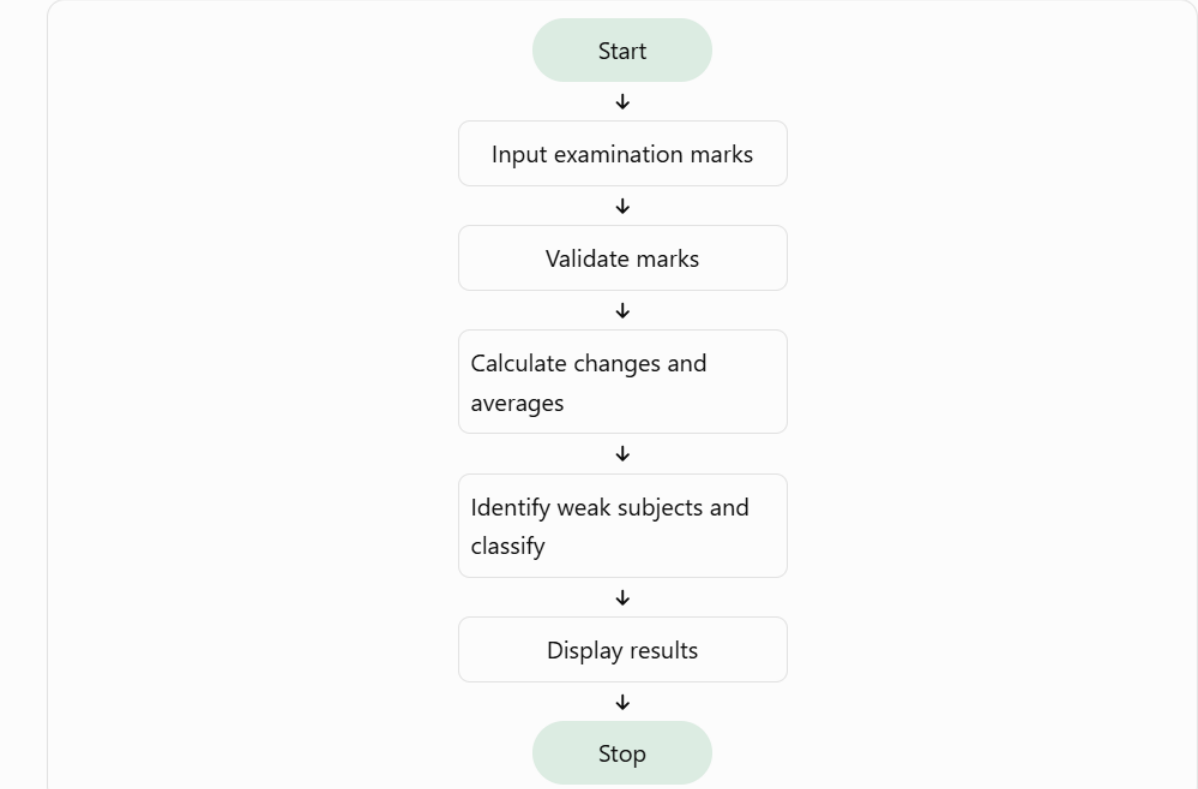
CALCULATE overall average

CLASSIFY performance

DISPLAY results

STOP

6. Flowchart



7. Python Source Code

```
subjects = ["Maths", "Science", "English"]
exam1 = []
exam2 = []

for subject in subjects:
    a = float(input(f"{subject} Exam 1 marks: "))
    b = float(input(f"{subject} Exam 2 marks: "))

    if not (0 <= a <= 100 and 0 <= b <= 100):
        print("Invalid marks")
        exit()

    exam1.append(a)
    exam2.append(b)

print("\nScore Analysis")
for i, subject in enumerate(subjects):
    change = exam2[i] - exam1[i]
    print(subject, "Change:", change)

weak = [subjects[i] for i in range(len(subjects))
        if exam1[i] < 40 and exam2[i] < 40]

average = sum(exam1 + exam2) / (len(subjects) * 2)

if average >= 80:
    grade = "Excellent"
elif average >= 60:
```

```
    grade = "Good"
elif average >= 40:
    grade = "Average"
else:
    grade = "Fail"


print("Overall Average:", round(average, 2))
print("Weak Subjects:", weak if weak else "None")
print("Classification:", grade)
```

9. Test Cases

Exam 1	Exam 2	Expected result
80, 70, 60	90, 75, 65	Improvement
30, 35, 70	35, 38, 75	Maths and Science weak
90, 85, 95	95, 90, 98	Excellent
110, 50, 60	—	Invalid marks

10. Sample Input and Output

```
Maths Exam 1 marks: 89
Maths Exam 2 marks: 99
Science Exam 1 marks: 89
Science Exam 2 marks: 78
English Exam 1 marks: 67
English Exam 2 marks: 66

Score Analysis
Maths Change: 10.0
Science Change: -11.0
English Change: -1.0
Overall Average: 81.33
Weak Subjects: None
Classification: Excellent
```

10. Conclusion

The program compares examination scores, calculates improvements and declines, identifies consistently weak subjects, and classifies students based on average performance.

4.Chemical Inventory Validation System – Python Project

1. Problem Statement

Develop a Python program to analyse hypothetical chemical inventory records, validate storage categories and maximum quantities, check chemical compatibility, and flag invalid combinations.

2. Objectives

- Validate chemical storage categories.
- Check maximum permitted quantities.
- Identify incompatible chemical combinations.
- Display inventory errors.

3. Assumptions

- Only hypothetical inventory records are used.
- Maximum quantity is 100 units per chemical.
- Acids and bases must be stored separately.
- Chemicals must use an approved storage category.

4. Algorithm

1. Start.
2. Input chemical names, categories, and quantities.
3. Validate storage categories.
4. Check maximum quantities.
5. Check compatibility restrictions.
6. Display errors or valid status.
7. Stop.

5. Pseudocode

START

INPUT chemical records

FOR each chemical:

CHECK storage category

CHECK quantity <= 100

IF invalid:

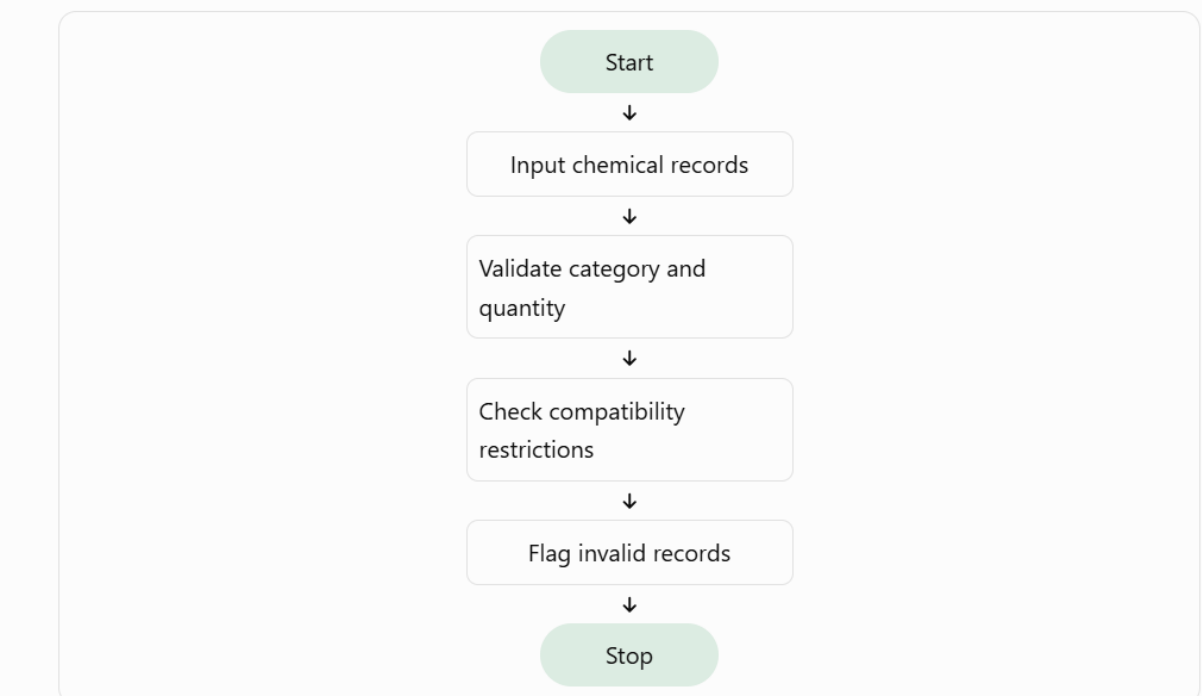
FLAG error

CHECK chemical compatibility

DISPLAY validation results

STOP

6. Flowchart



7.Python Source Code

```
chemicals = [
  ("Chemical A", "Acid", 50),
  ("Chemical B", "Base", 40),
  ("Chemical C", "Neutral", 120)
]

allowed_categories = {"Acid", "Base", "Neutral"}
incompatible = {("Acid", "Base"), ("Base", "Acid")}

for name, category, quantity in chemicals:
  errors = []
```

```
    if category not in allowed_categories:
        errors.append("Invalid category")

    if quantity < 0 or quantity > 100:
        errors.append("Invalid quantity")

    for other_name, other_category, _ in chemicals:
        if name != other_name and (
            category, other_category
        ) in incompatible:
            errors.append(
                f"Incompatible with {other_name}"
            )

    if errors:
        print(name, ":", "; ".join(errors))
    else:
        print(name, ": Valid")
```

8. **Test Cases**

Chemical category	Quantity	Expected result
Acid	50	Valid
Base	40	Compatibility warning with acid
Neutral	120	Quantity exceeded
Unknown	20	Invalid category

9. Sample Output

```
Chemical A : Incompatible with Chemical B
Chemical B : Incompatible with Chemical A
Chemical C : Invalid quantity
```

10. Conclusion

The program checks hypothetical chemical inventories for category errors, quantity violations, and incompatible combinations. Actual storage decisions must follow the relevant safety data sheets and applicable chemical-storage regulations.

5.Electricity Bill Calculation and Usage Analysis – Python Project

1. Problem Statement

Develop a Python program to calculate electricity bills for multiple households using consumption slabs, fixed fees, and penalties, compare consumption patterns, and identify excessive usage.

2. Objectives

- Calculate bills using consumption slabs.
- Add fixed fees and penalties.
- Compare household consumption.
- Identify excessive electricity usage.

3. Assumptions

- First 100 units: ₹2/unit.
- Next 100 units: ₹3/unit.
- Above 200 units: ₹5/unit.
- Fixed fee: ₹50.
- Penalty: ₹100 if usage exceeds 300 units.
- Excessive usage: above 250 units.

4. Algorithm

1. Start.
2. Input the number of households.
3. Enter units consumed by each household.
4. Calculate slab-wise charges.
5. Add fixed fee and applicable penalty.
6. Identify excessive usage.
7. Compare household consumption and display bills.
8. Stop.

5. Pseudocode

START

INPUT number_of_households

FOR each household:

  INPUT units

  IF units <= 100:

    bill = units * 2

  ELSE IF units <= 200:

    bill = 200 + (units - 100) * 3

  ELSE:

    bill = 500 + (units - 200) * 5

  bill = bill + 50

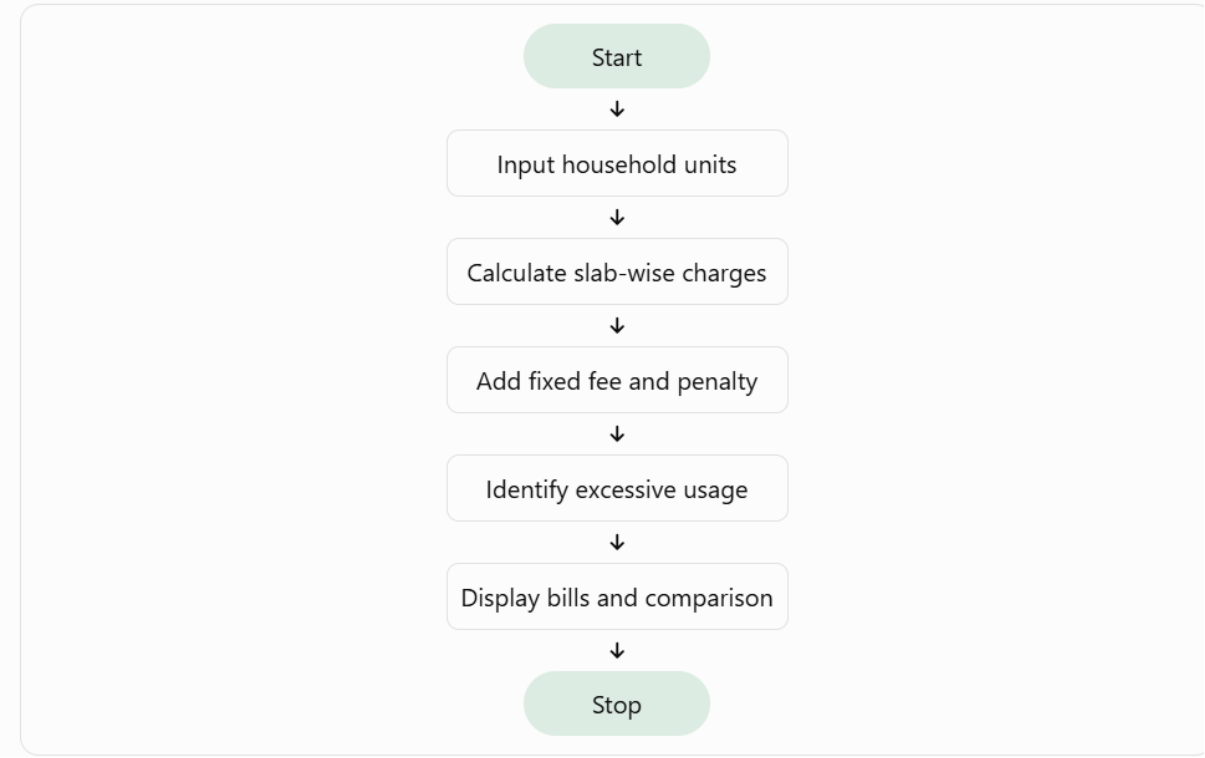
  IF units > 300:

    bill = bill + 100

  DISPLAY bill and usage status

STOP

6. Flowchart



7. Python Source Code

```
n = int(input("Number of households: "))
records = []

for i in range(n):
    units = int(input(f"Household {i+1} units: "))

    if units < 0:
        print("Invalid units")
        continue

    if units <= 100:
        bill = units * 2
    elif units <= 200:
        bill = 200 + (units - 100) * 3
    else:
        bill = 500 + (units - 200) * 5

    bill += 50

    if units > 300:
        bill += 100

    status = "Excessive" if units > 250 else "Normal"
    records.append((i + 1, units, bill))

    print(f"Household {i+1}: Bill = Rs.{bill}, {status}")
```

if records:

highest = max(records, key=lambda x: x[1])

print("Highest consumption: Household", highest[0])

9. Test Cases

Units	Expected bill	Usage
50	₹150	Normal
150	₹400	Normal
250	₹800	Normal
350	₹1,350	Excessive

10. Sample Input and Output

```
Number of households: 3
Household 1 units: 300
Household 1: Bill = Rs.1050, Excessive
Household 2 units: 500
Household 2: Bill = Rs.2150, Excessive
Household 3 units: 600
Household 3: Bill = Rs.2650, Excessive
Highest consumption: Household 3
```

10. Conclusion

The program calculates household electricity bills using consumption slabs, fixed fees, and penalties. It also compares usage and identifies households with excessive consumption.

6.Monthly Expense Analysis System – Python Project

1. Problem Statement

Develop a Python program to analyse nested lists of monthly expenses across branches and categories, calculate totals and averages, and identify the largest contributors to expenditure.

2. Objectives

- Calculate branch-wise expenses.
- Find monthly totals and averages.
- Identify the highest expense category.

- Compare expenditure across branches.

3. Assumptions

- Expenses are stored in nested lists.
- Each branch has three categories: Rent, Salary, and Utilities.
- All expenses are non-negative.
- Data represents three months.

4. Algorithm

1. Start.
2. Store monthly expenses in nested lists.
3. Calculate branch-wise totals and averages.
4. Calculate category-wise totals.
5. Identify the largest expense contributor.
6. Display results.
7. Stop.

5. Pseudocode

START

STORE expenses in nested lists

FOR each branch:

  CALCULATE total and average

FOR each category:

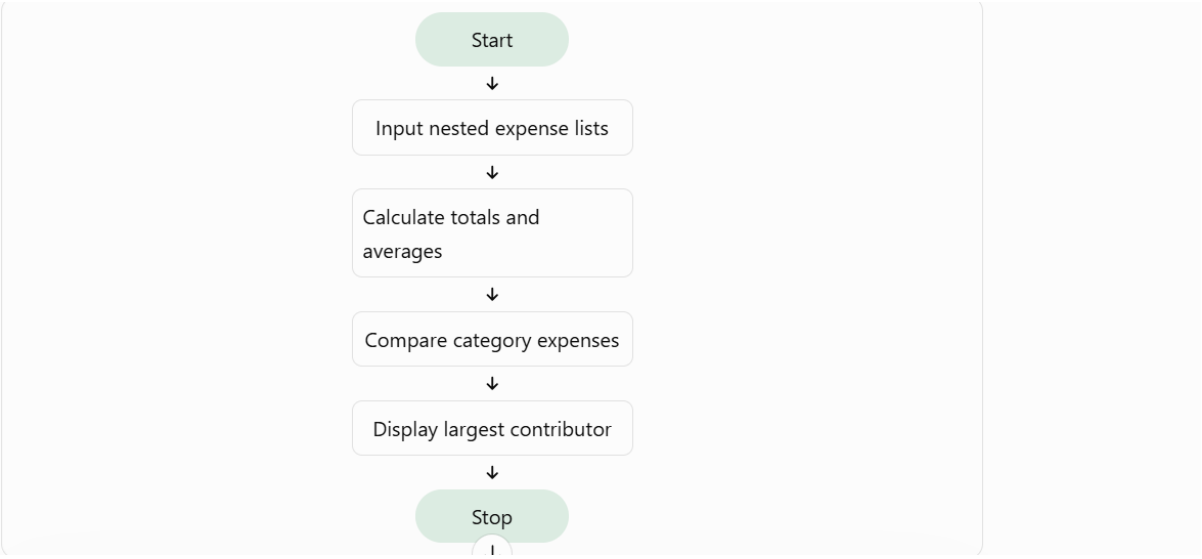
  CALCULATE category total

IDENTIFY largest contributor

DISPLAY results

STOP

6. Flowchart



7.Python Source Code

```
branches = {
  "Branch A": [
    [1000, 2000, 500],
    [1200, 2200, 600],
    [1100, 2100, 550]
  ],
  "Branch B": [
    [1500, 2500, 700],
    [1400, 2400, 650],
    [1600, 2600, 750]
  ]
}

categories = ["Rent", "Salary", "Utilities"]
category_totals = [0, 0, 0]

for branch, months in branches.items():
  total = sum(sum(month) for month in months)
  average = total / len(months)
  print(branch, "Total:", total,
      "Monthly Average:", round(average, 2))

  for month in months:
    for i, amount in enumerate(month):
      category_totals[i] += amount
```

```
print("\nCategory Totals:")
for i, category in enumerate(categories):
    print(category, ":", category_totals[i])

largest = max(range(len(categories)),
              key=lambda i: category_totals[i])
print("Largest contributor:", categories[largest])
```

8. **Test Cases**

Input	Expected result
Branch A expenses	Total ₹11,250
Branch B expenses	Total ₹16,100
All branch expenses	₹27,350
Category comparison	Salary is highest

9. **Sample Output**

```
Branch A Total: 11250 Monthly Average: 3750.0
Branch B Total: 14100 Monthly Average: 4700.0

Category Totals:
Rent : 7800
Salary : 13800
Utilities : 3750
Largest contributor: Salary
```

10. Conclusion

The program analyses monthly expenses using nested lists, calculates branch totals and averages, and identifies the category contributing the most to expenditure.

7.Smart Canteen Order and Stock Management

1. Problem Statement

To process customer orders, calculate bills, update inventory, reject orders exceeding available stock, and summarize daily sales and remaining stock.

2. Objectives

- Calculate bills for customer orders.
- Update stock after successful orders.

- Reject unavailable quantities.
- Display total daily sales and remaining inventory.

3. Assumptions

- Product prices and initial stock are predefined.
- Customers enter valid product names and quantities.
- Only available quantities are sold.
- Daily sales include only successful orders.

4. Algorithm

1. Initialize product prices and stock.
2. Read customer orders.
3. Check product availability and quantity.
4. If available, calculate bill and update stock.
5. Otherwise, reject the order.
6. Display total sales and remaining stock.

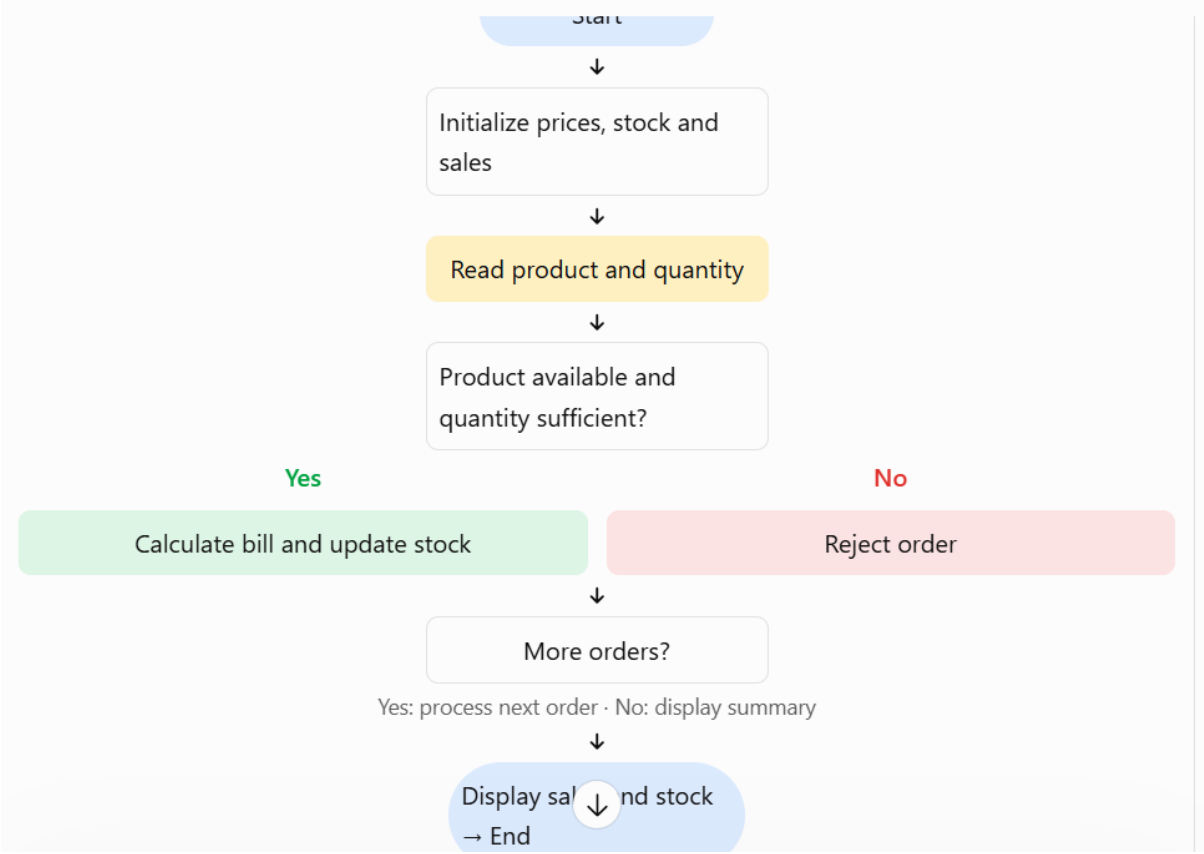
5. Pseudocode

```
START
Initialize prices, stock, total_sales = 0
WHILE more orders exist
  Read product and quantity
  IF product exists AND quantity <= stock
    bill = price * quantity
    stock = stock - quantity
    total_sales = total_sales + bill
    Display bill
  ELSE
    Display "Order Rejected"
  END IF
END WHILE
```

Display total_sales and remaining stock

STOP

7. **Flowchart**



8. **Python Source Code**

```
prices = {"Pen": 10, "Book": 50, "Bag": 500}
stock = {"Pen": 20, "Book": 10, "Bag": 5}
total_sales = 0

orders = [("Pen", 3), ("Book", 2), ("Bag", 6)]

for product, qty in orders:
  if product in stock and qty > 0 and qty <= stock[product]:
    bill = prices[product] * qty
    stock[product] -= qty
    total_sales += bill
    print(product, "Bill: Rs.", bill)
  else:
    print(product, "- Order Rejected")

print("Total Sales: Rs.", total_sales)
```

print("Remaining Stock:", stock)

9. **Test Cases**

Product	Quantity	Expected result
Pen	3	Bill ₹30
Book	2	Bill ₹100
Bag	6	Rejected
Pencil	1	Rejected (unknown product)

9. Sample Output

```
Pen Bill: Rs. 30
Book Bill: Rs. 100
Bag - Order Rejected
Total Sales: Rs. 130
Remaining Stock: {'Pen': 17, 'Book': 8, 'Bag': 5}
```

10. Conclusion

The program processes orders, calculates bills, rejects invalid or unavailable quantities, updates inventory, and summarizes daily sales and remaining stock.

8.Multi-Event Sports Scheduling and Scoring

1. Problem Statement

To schedule multiple sports events, detect scheduling conflicts, calculate team scores, aggregate points, and generate final team rankings using Python lists and tuples.

2. Objectives

- Store event schedules using lists and tuples.
- Identify overlapping event schedules.
- Calculate total points for each team.
- Display teams in ranking order.

3. Assumptions

- Each event has a name, time slot, and participating teams.
- Events at the same time cannot use the same venue.
- Team scores are non-negative integers.
- Higher total points mean a better rank.

4. Algorithm

1. Store event schedules and scores.
2. Check events for scheduling conflicts.
3. Calculate total points for each team.
4. Sort teams by total points in descending order.
5. Display conflicts and final rankings.

5. Pseudocode

START

Store event schedules and team scores

FOR each pair of events

  IF time and venue are the same

    Display scheduling conflict

FOR each team

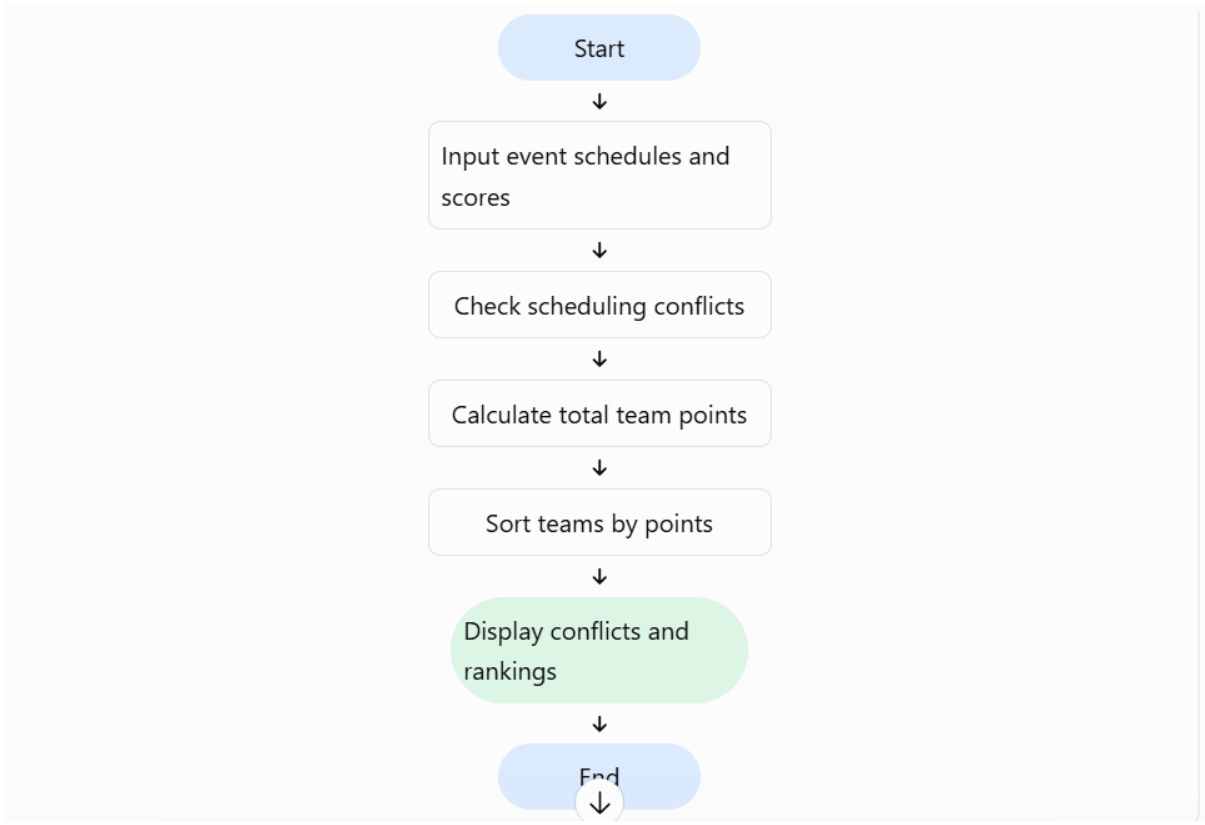
  Calculate total points

Sort teams by total points descending

Display final rankings

STOP

6. Flowchart



7.**Python Source Code**

```
events = [
  ("Football", "10:00", "Ground A"),
  ("Basketball", "10:00", "Ground A"),
  ("Tennis", "11:00", "Court B")
]

scores = [
  ("Team A", 30),
  ("Team B", 45),
  ("Team C", 25),
  ("Team A", 20)
]

print("Scheduling Conflicts:")
```

```
conflict_found = False

for i in range(len(events)):
    for j in range(i + 1, len(events)):
        if (events[i][1] == events[j][1]
                and events[i][2] == events[j][2]):
            print(events[i][0], "and", events[j][0])
            conflict_found = True

if not conflict_found:
    print("No conflicts")

points = {}
for team, score in scores:
    points[team] = points.get(team, 0) + score

rankings = sorted(
    points.items(), key=lambda x: x[1], reverse=True
)

print("\nFinal Rankings:")
for rank, (team, total) in enumerate(rankings, 1):
    print(rank, team, "-", total, "points")
```

7. Test Cases

Test case	Input/condition	Expected result
1	Two events, same time and venue	Conflict detected
2	Team A scores 30 and 20	Total = 50
3	Team B scores 45	Total = 45
4	Different event times	No conflict

10. Sample Output

```
Scheduling Conflicts:
Football and Basketball

Final Rankings:
1 Team A - 50 points
2 Team B - 45 points
3 Team C - 25 points
```

10. Conclusion

The program uses Python lists and tuples to manage sports schedules, identify venue conflicts, calculate team totals, and generate final rankings.

9.University Academic Progression Audit

1. Problem Statement

To evaluate students' academic progression by checking semester marks, subject pass conditions, attendance, and prerequisite completion, and identify outstanding requirements.

2. Objectives

- Check subject marks and pass status.
- Verify minimum attendance.
- Check prerequisite completion.
- Determine progression eligibility.
- Display outstanding requirements.

3. Assumptions

- Minimum passing mark is 40.
- Minimum attendance required is 75%.
- All prerequisite records are available.

- Students must satisfy all conditions for progression.

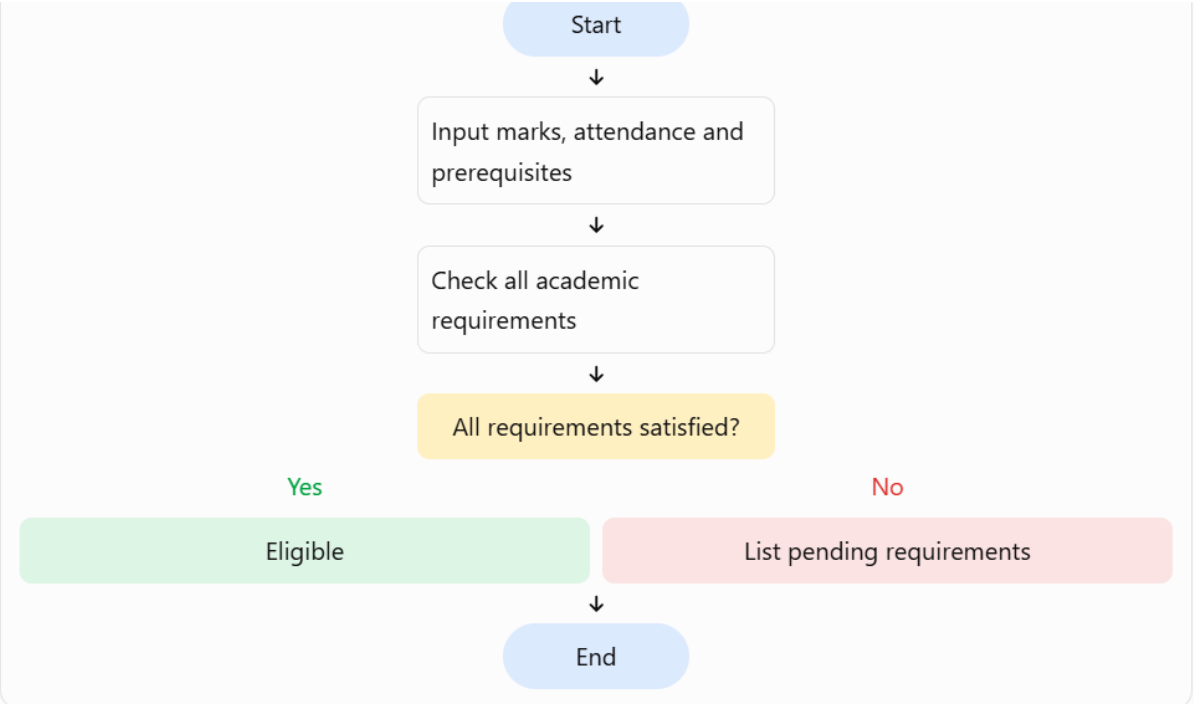
4. Algorithm

1. Input student marks, attendance, and prerequisite status.
2. Check whether each subject meets the pass mark.
3. Verify attendance requirements.
4. Check prerequisite completion.
5. Display eligibility and pending requirements.

5. Pseudocode

```
START
Input marks, attendance, prerequisites
Set eligible = TRUE
IF any subject mark < 40
   Report failed subjects
   Set eligible = FALSE
IF attendance < 75
   Report attendance shortage
   Set eligible = FALSE
IF prerequisites incomplete
   Report pending prerequisites
   Set eligible = FALSE
IF eligible
   Display "Eligible for Progression"
ELSE
   Display "Not Eligible"
STOP
```

6. **Flowchart**



7.**Python Source Code**

```
marks = {"Maths": 75, "Physics": 35, "Python": 80}
attendance = 82
prerequisites = {"Maths": True, "Physics": True, "Python": False}

pending = []

for subject, mark in marks.items():
  if mark < 40:
    pending.append(subject + ": Failed")

if attendance < 75:
  pending.append("Attendance shortage")

for subject, completed in prerequisites.items():
  if not completed:
    pending.append(subject + ": Prerequisite incomplete")

if pending:
  print("Not Eligible for Progression")
  print("Outstanding Requirements:")
  for item in pending:
    print("-", item)
```

```
else:
    print("Eligible for Progression")
```

8. **Test Cases**

Test	Marks	Attendance	Prerequisites	Expected result
1	All ≥ 40	85%	Complete	Eligible
2	One mark = 35	82%	Complete	Not eligible
3	All ≥ 40	65%	Complete	Not eligible
4	All ≥ 40	90%	Incomplete	Not eligible

9. **Sample Output**

```
Not Eligible for Progression
Outstanding Requirements:
- Physics: Failed
- Python: Prerequisite incomplete
```

10. Conclusion

The program audits student academic records, checks progression conditions, and reports eligibility along with outstanding requirements.

10.Campus Event Seating and Entry Validation:

1. Problem Statement

To manage campus event seating using nested lists, validate tickets with registration IDs, detect duplicate entries, and calculate available seating capacity.

2. Objectives

- Generate seating assignments.
- Validate tickets against registered IDs.
- Detect duplicate entries.
- Calculate occupied and available seats.

3. Assumptions

- Each seat has a unique identifier.
- Registered IDs are predefined.
- Each valid registration can enter only once.

- Seating capacity is fixed.

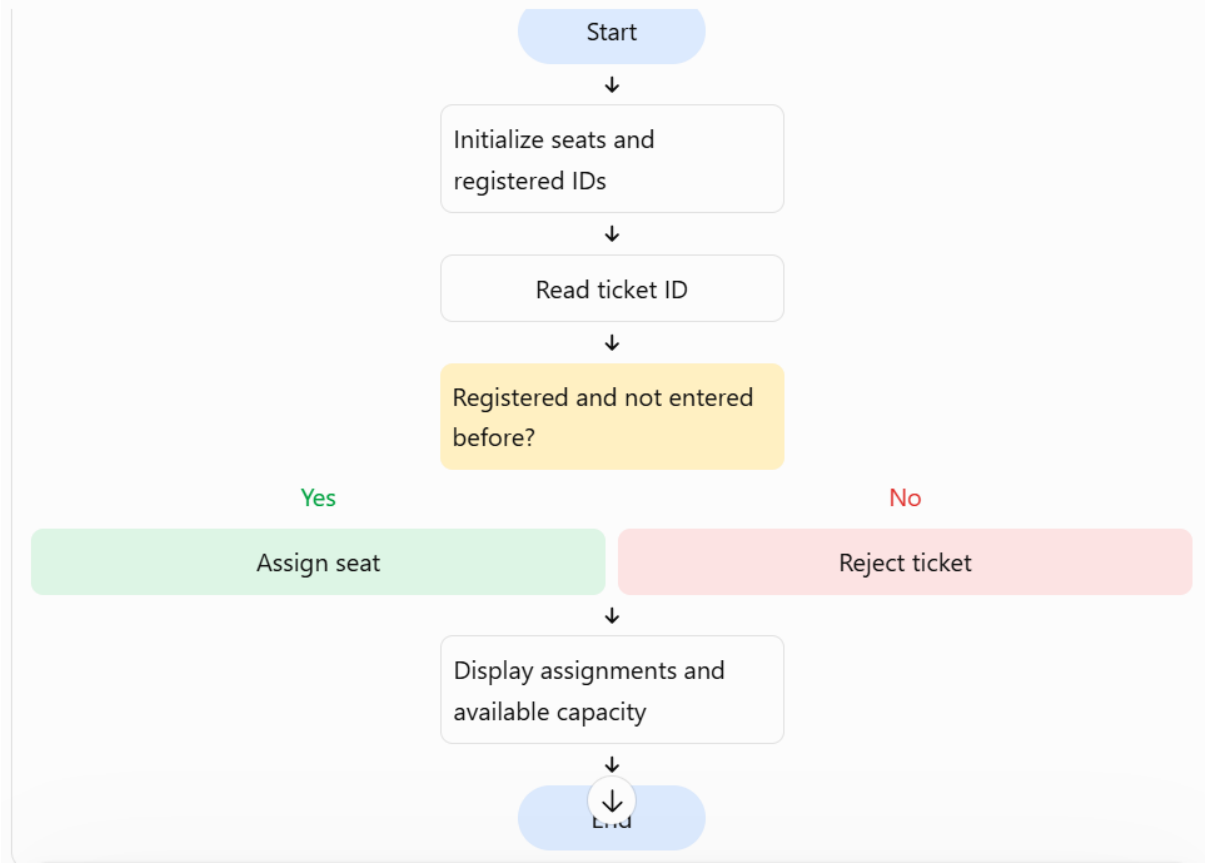
4. Algorithm

1. Create a nested list of seats.
2. Store registered IDs and ticket entries.
3. Validate each ticket ID.
4. Reject invalid and duplicate entries.
5. Assign seats to valid attendees.
6. Display occupied and available seats.

5. Pseudocode

```
START
Create seating list and registered IDs
Set entered = empty set
FOR each ticket ID
  IF ID is not registered
    Display Invalid Ticket
  ELSE IF ID already entered
    Display Duplicate Entry
  ELSE
    Assign available seat
    Add ID to entered
  END IF
END FOR
Calculate available seats
Display assignments and capacity
STOP
```

7. Flowchart



7.**Python Source Code**

```
Nested list representing 2 rows with 3 seats each
seats = [[None] * 3 for _ in range(2)]

registered = {"R101", "R102", "R103", "R104"}
tickets = ["R101", "R102", "R101", "R999", "R103"]
entered = set()

for ticket in tickets:
  if ticket not in registered:
    print(ticket, "- Invalid Ticket")
  elif ticket in entered:
    print(ticket, "- Duplicate Entry")
  else:
    assigned = False
    for i in range(len(seats)):
      for j in range(len(seats[i])):
        if seats[i][j] is None:
          seats[i][j] = ticket
          entered.add(ticket)
          print(ticket, "- Seat", i + 1, j + 1)
```

```
                assigned = True
                break
        if assigned:
            break

capacity = sum(len(row) for row in seats)
occupied = len(entered)
print("Occupied Seats:", occupied)
print("Available Seats:", capacity - occupied)
```

8. **Test Cases**

Test input	Expected result
R101	Valid entry
R101 entered again	Duplicate entry
R999	Invalid ticket
3 valid unique IDs	3 seats occupied

9. **Sample Output**

```
R101 - Seat 1 1
R102 - Seat 1 2
R101 - Duplicate Entry
R999 - Invalid Ticket
R103 - Seat 1 3
Occupied Seats: 3
Available Seats: 3
```

10. Conclusion

The program assigns seats, validates registrations, detects duplicate and invalid tickets, and reports remaining seating capacity.